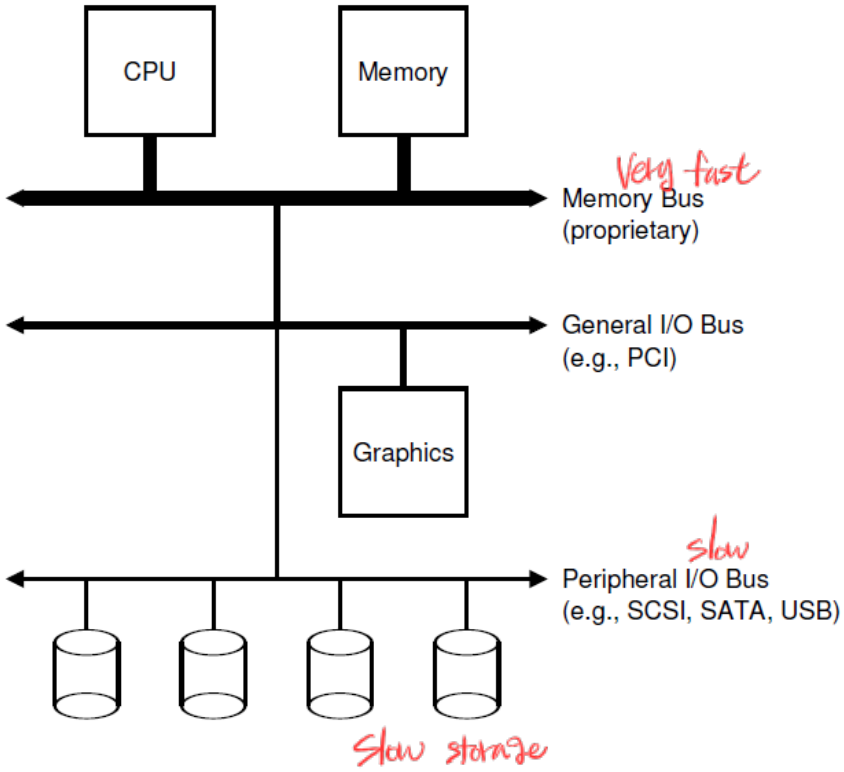
I/O Devices & SSD

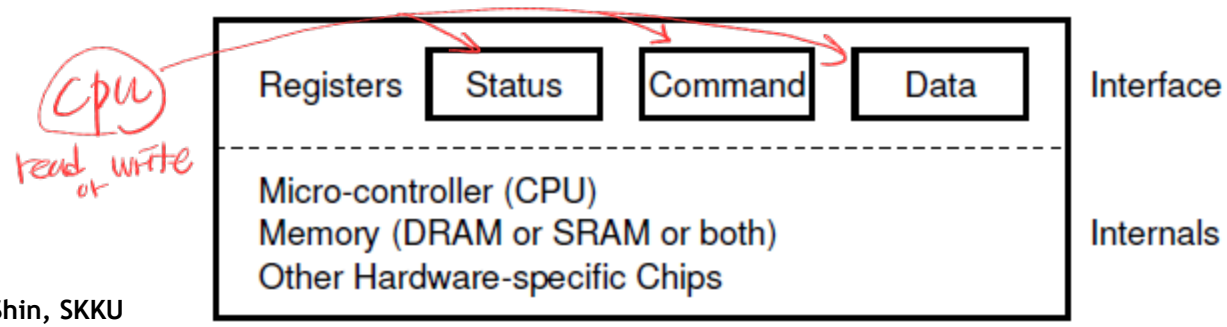
System Architecture

- **Hierarchical approach**
- **Memory bus**
  - CPU and memory
  - Fastest
- **I/O bus**
  - e.g., PCI
  - Graphics and higher-performance I/O devices
- **Peripheral bus**
  - SCSI, SATA, or USB
  - Connect many slowest devices



Device Structure

- **Hardware Interface**
  - Allows the system software to control its operation
  - status register: can be read to see the current status of the device
  - command register: can be written to tell the device to perform a certain task
  - data register: transfer data to/from the device
- **Internal Structure**
  - Implementation specific
  - Simple devices
    - have one or a few hardware chips to implement their functionality;
  - Complex devices
    - include a simple CPU running firmware, some general purpose memory, and other device-specific chips



Protocol

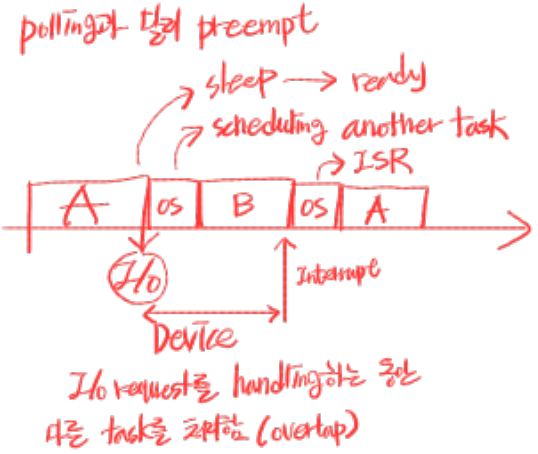
- By reading and writing internal registers of a device, the operating system can control device behavior.

```
While (STATUS == BUSY)
        ; // wait until device is not busy -> polling
Write data to DATA register
Write command to COMMAND register
        (Doing so starts the device and executes the command)
While (STATUS == BUSY)
        ; // wait until device is done with your request
```

- Inefficiencies and Inconveniences
  - Polling
    - Repeatedly reading the status register
  - Programmed I/O (PIO)
    - The main CPU is involved with the data movement

Lowering CPU Overhead With Interrupts

- **Interrupt**
  - OS can issue a request, put the calling process to sleep, and context switch to another task.
  - When the device is finally finished with the operation, it will raise a hardware interrupt, causing the CPU to jump into the OS at a pre-determined interrupt service routine (ISR)
  - The handler in operating system code that will finish the request
    - e.g., reading data and perhaps an error code from the device
  - Wake the process waiting for the I/O
  - Allow for overlap of computation and I/O
- Interrupt is not *always* the best solution
  - If a device is fast, it may be best to poll.
  - Otherwise, use interrupt



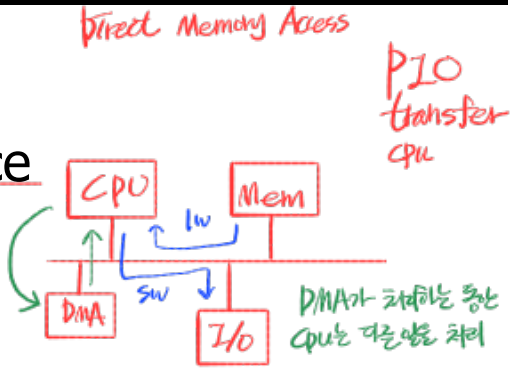
polling

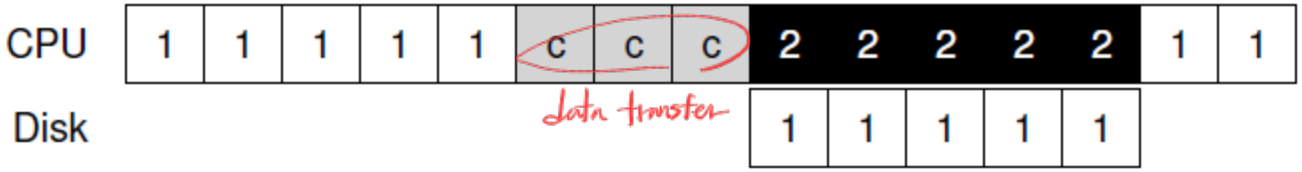
Interrupt Optimization

- **Unknown device: hybrid approach**
  - polls for a little while and then, if the device is not yet finished, uses interrupts. (two-phased approach)
- **Network systems**
  - When a huge stream of incoming packets each generate an interrupt, it is possible for the OS to livelock
    - only processing interrupts and never allowing a user-level process to run
  - Occasionally use polling to better control what is happening in the system (e.g., slashdot effect)
- **Interrupt Coalescing**
  - Multiple interrupts are coalesced into a single interrupt delivery
    - lower the overhead of interrupt processing
  - Long waiting
    - Many interrupts can be coalesced
    - Increase the latency of a request

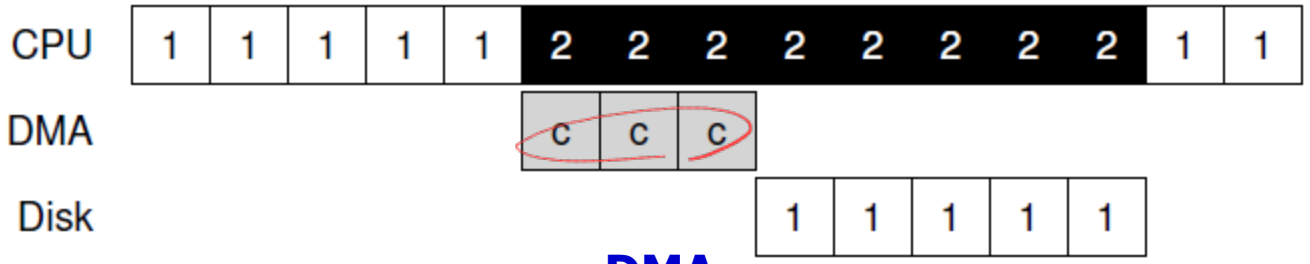
More Efficient Data Movement With DMA

- OS programs the DMA engine
  - Source/destination address in memory or device
  - Data amount
- Other processes can be serviced
- When the DMA is complete, the DMA controller raises an interrupt, and the OS thus knows the transfer is complete.





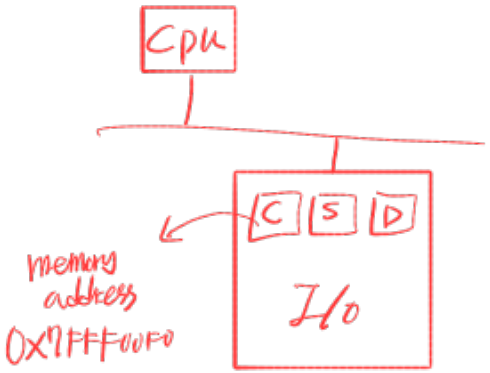
Programmed IO

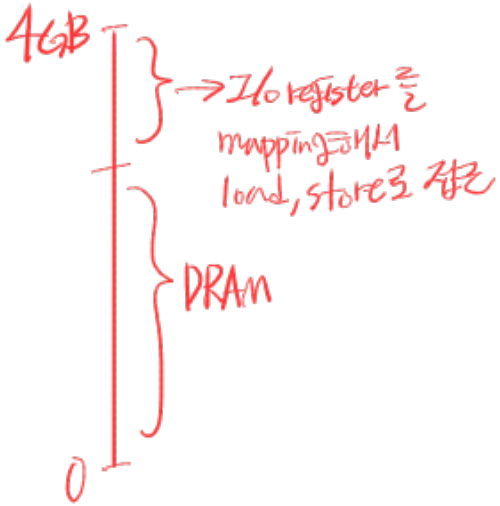


DMA

Methods Of Device Interaction

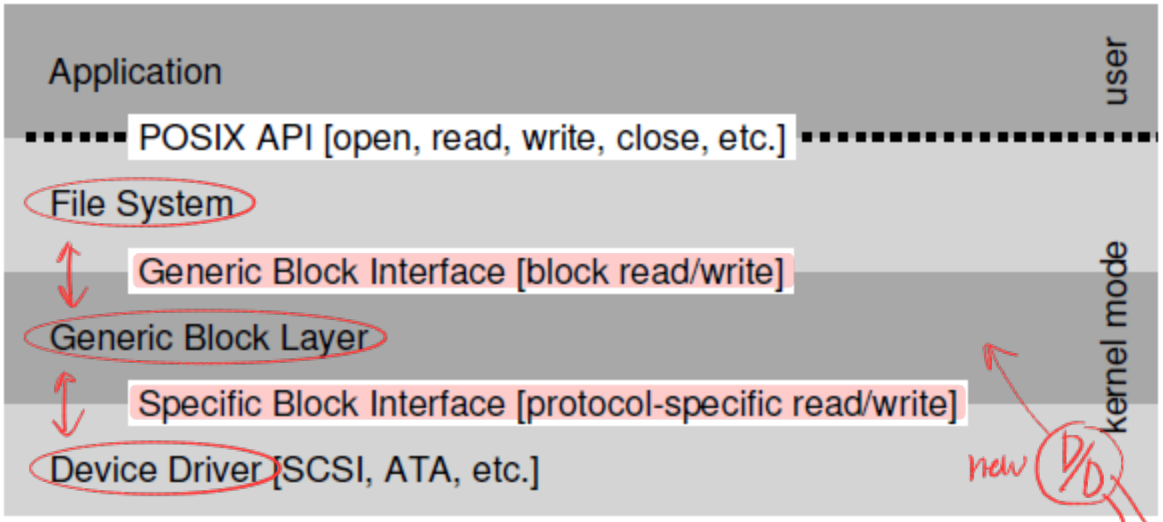
- **I/O instructions**
  - E.g., in and out on x86
  - To send data to a device, register → **port**.
  - Usually privileged instructions
- **Memory-mapped I/O**
  - Device registers are mapped to memory address space
  - load (to read) or store (to write)





Device Driver

- How can we keep most of the OS device-neutral, thus hiding the details of device interactions from major OS subsystems?
- Solution: **Abstraction**
  - The device driver in the OS know in detail how a device works
  - Any specifics of device interaction are encapsulated within D/D



Issues block requests to the generic block layer

Routes block requests to the appropriate D/D

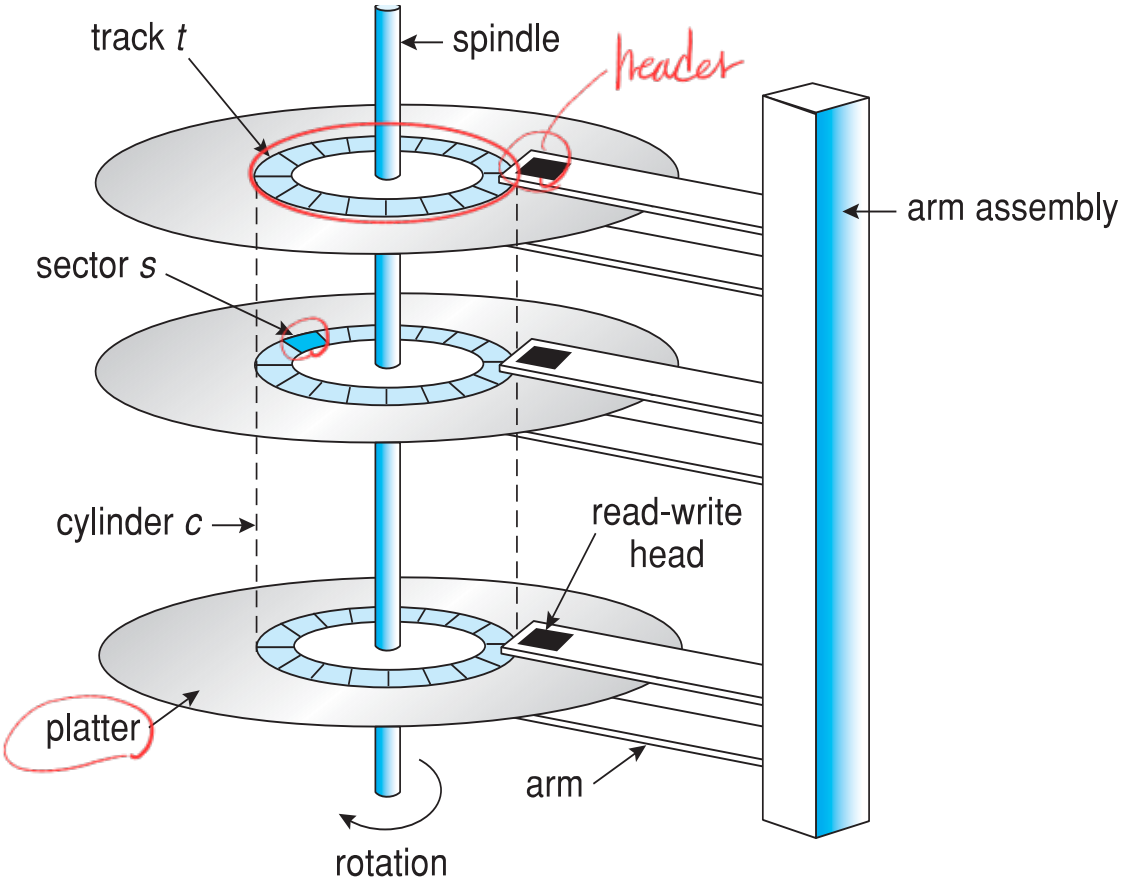
handles the details of issuing the specific request

File System Stack

Device Driver

- Drawbacks of encapsulation
  - Even if a device has many special capabilities, it has to present a generic interface to the rest of the kernel, those special capabilities will go unused.
  - e.g., SCSI devices which have very rich error reporting
- Over 70% of OS code is found in device drivers
  - for any given installation, most of that code may not be active
  - have many more bugs and thus are a primary contributor to kernel crashes

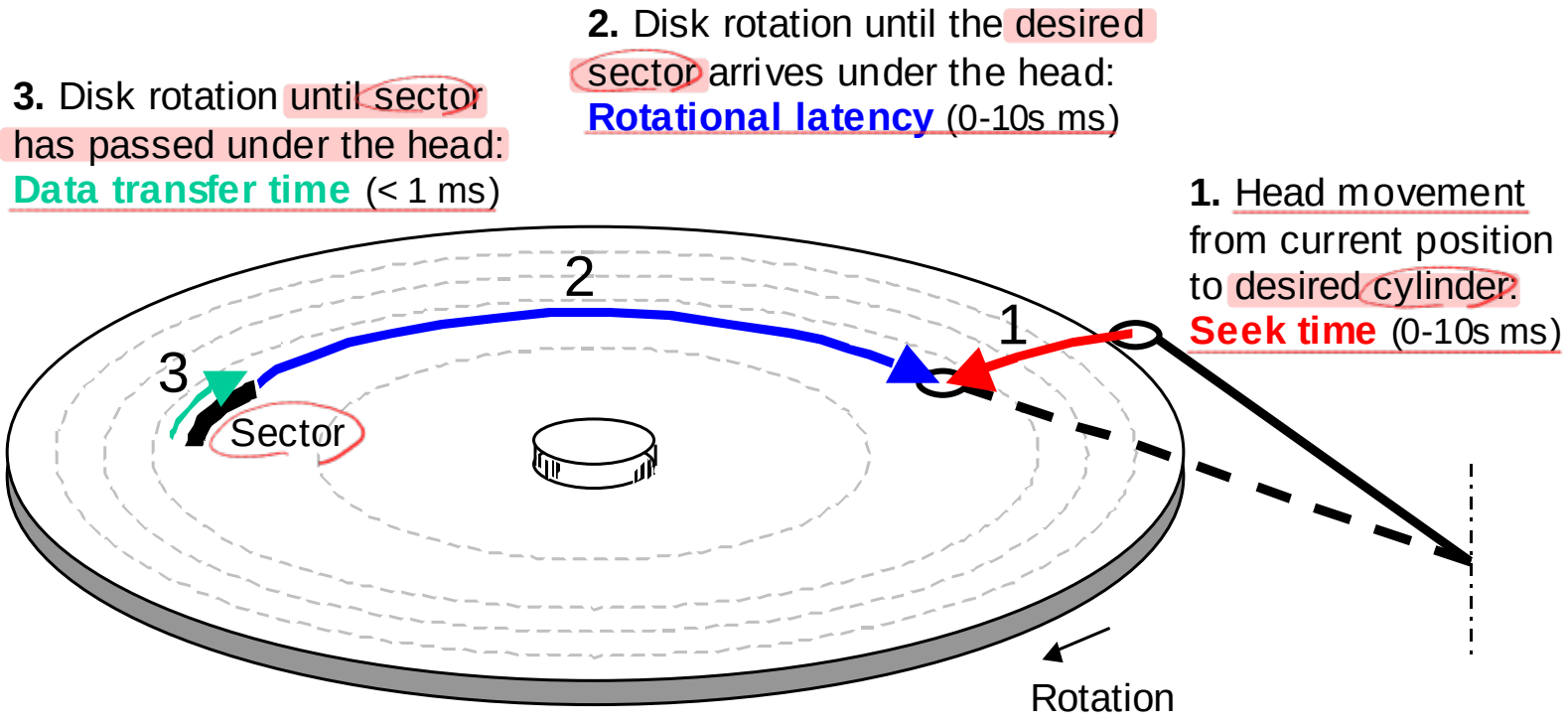
Disk System


track t
spindle
header
arm assembly
sector s
cylinder c
read-write
head
platter
arm
rotation

Disk System

- **Data access in disk system**
  - Seek time
  - Rotational delay (latency time)
  - Data transmission time

1, cylinder를 찾고 (seek time)
2, sector를 찾고 (rotational latency)
3, head가 sector를 읽는다 (data transfer time)

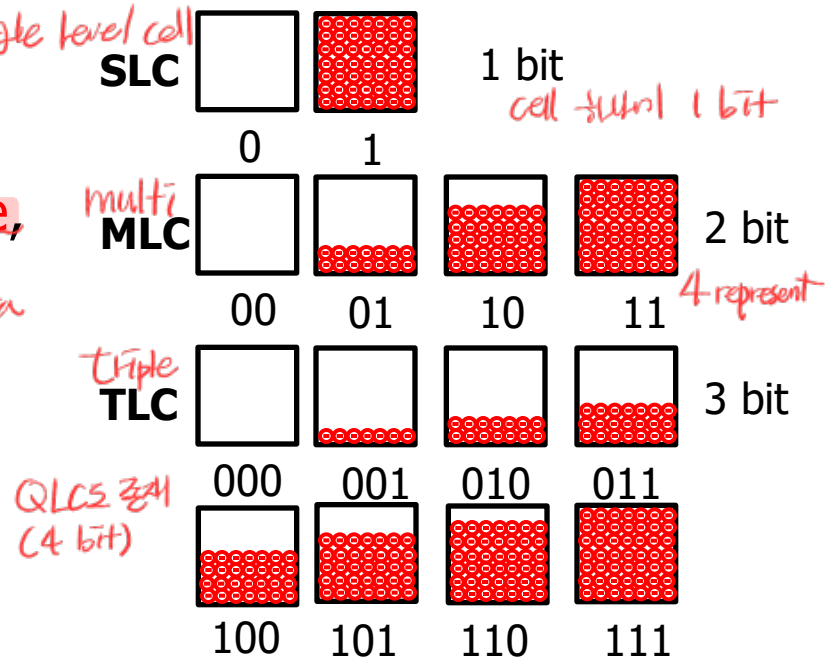


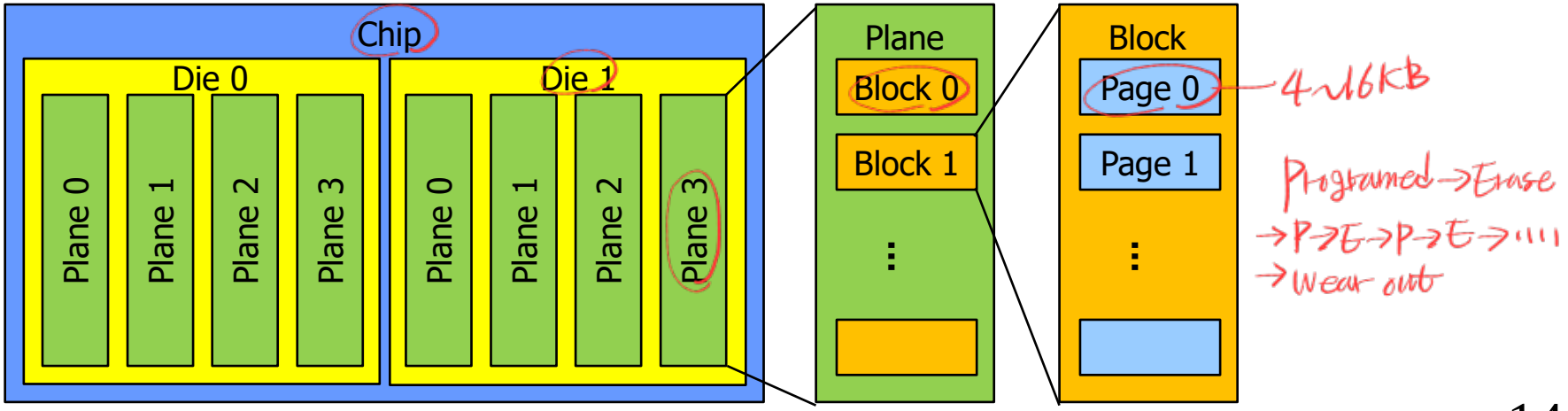
Disk System

- **Average time to access some target sector**
  - $T_{access} = T_{avg\text{-}seek} + T_{avg\text{-}rotation} + T_{avg\text{-}transfer}$
    - Seek time ($T_{avg\text{-}seek}$)
      - Time to position heads over cylinder containing target sector
      - Typical $T_{avg\text{-}seek}$ is 3~9ms
    - Rotational latency ($T_{avg\text{-}rotation}$)
      - Time waiting for first bit of target sector to pass under head
      - $T_{avg\text{-}rotation} = 1/2 \times 1/RPMs \times 60s/1min$
      - Typical $T_{avg\text{-}rotation} = 1\sim4ms$
    - Transfer time ($T_{avg\text{-}transfer}$)
      - Time to read the bits in the target sector
      - $T_{avg\text{-}transfer} = 1/RPM \times 1/(avg.\ \#\ sectors/track) \times 60s/1min$

Flash-based SSD

- **NAND flash memory**
  - Page: read/write unit, 4KB~16KB
  - Block: erase unit, erase-before-write, 128 KB or 256 KB
  - Limited lifetime: wear out
  - SLC, MLC, TLC

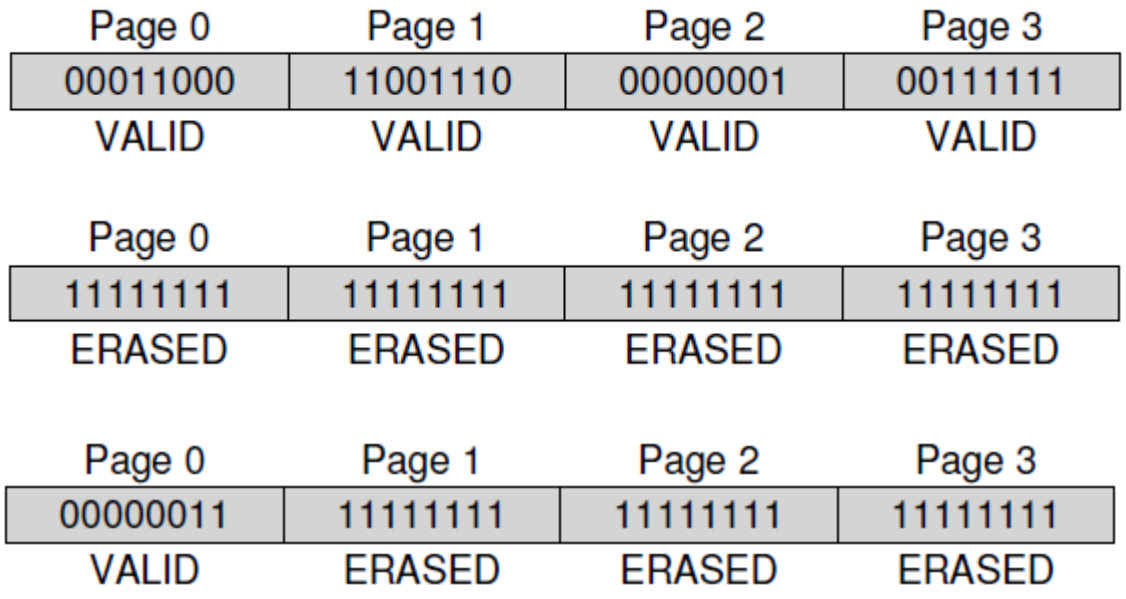




Flash Operations

- **Read (a page)**
  - Random access
- **Erase (a block)**
  - set each bit to the value 1
- **Program (a page)**
  - change some of the 1's within a page to 0's

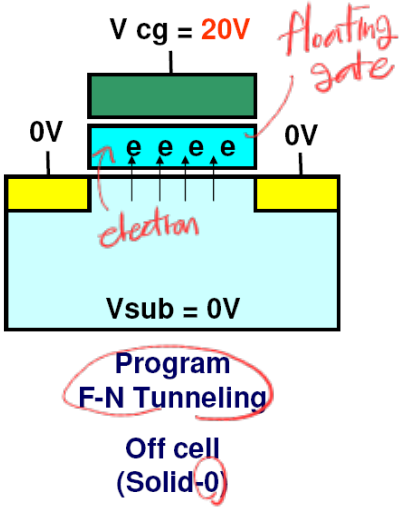
Device	Read (μs)	Program (μs)	Erase (μs)
SLC	25	200-300	1500-2000
MLC	50	600-900	~3000
TLC	~75	~900-1350	~4500

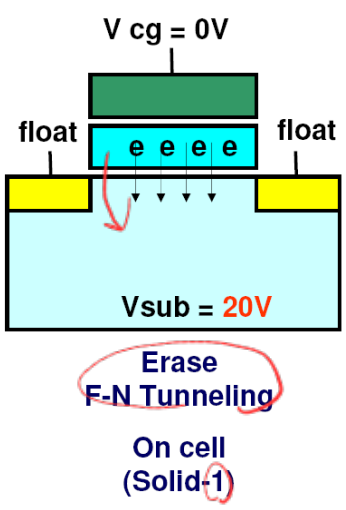


Previous contents of Page 1, 2, 3 are all gone!
They must be moved before the block erase operation.

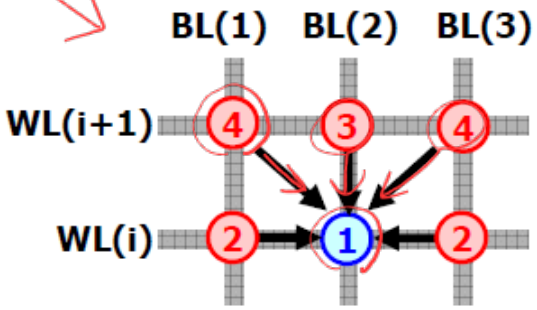
Flash Performance And Reliability

- **Wear out**
  - Erased/program slowly accrues a little bit of extra charge.
  - As that extra charge builds up, the block becomes unusable.
  - MLC block: 10,000 P/E (Program/Erase) cycle lifetime;
  - SLC block: 100,000 P/E cycles.
- **Disturbance**
  - When accessing a particular page within a flash, it is possible that some bits get flipped in neighboring pages
  - read disturbs or program disturbs





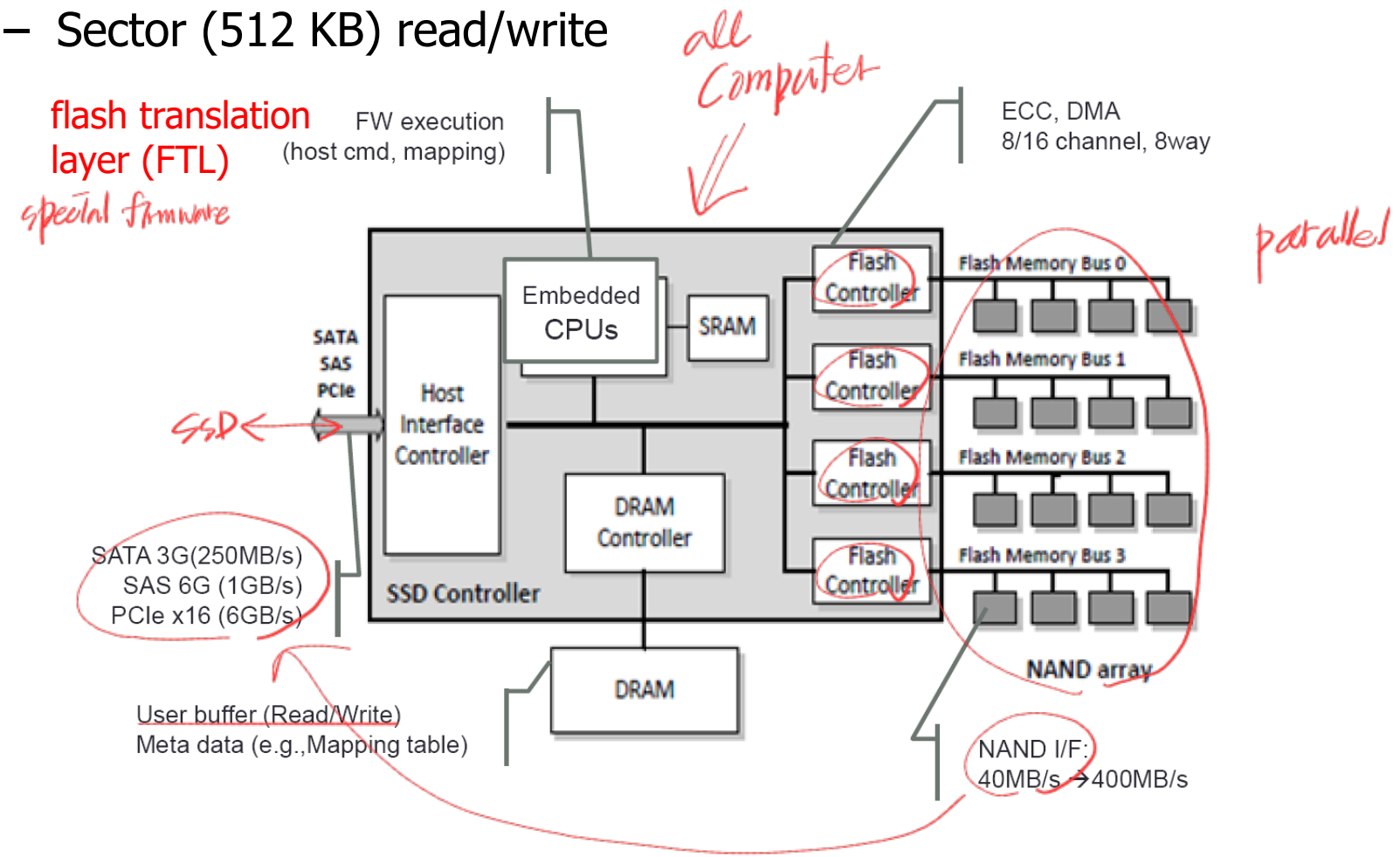
If you pass water droplets through thin paper many times, …



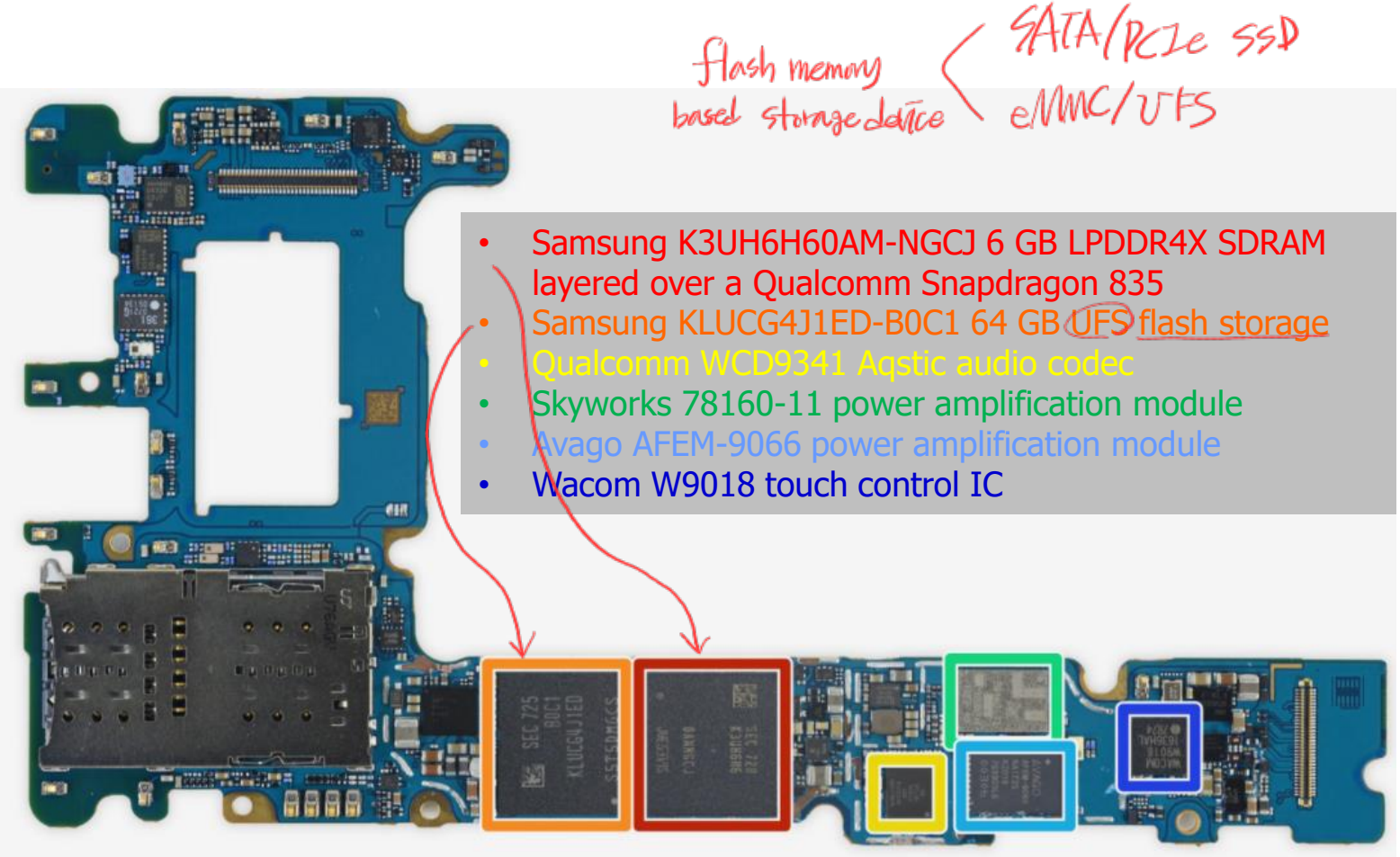
cell-to-cell interference

From Raw Flash to Flash-Based SSDs

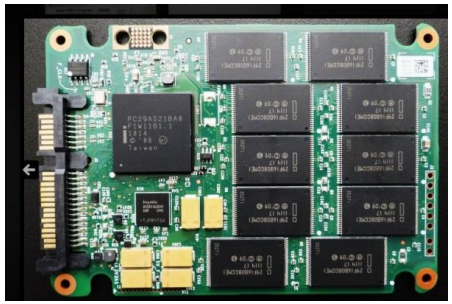
- Flash-based SSD provides standard block interface atop the raw flash chips inside it.
  - Sector (512 KB) read/write



Samsung Galaxy Note8



Solid-State Disk



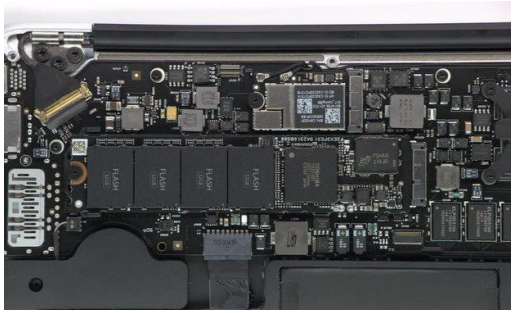
SATA SSD

PCIe

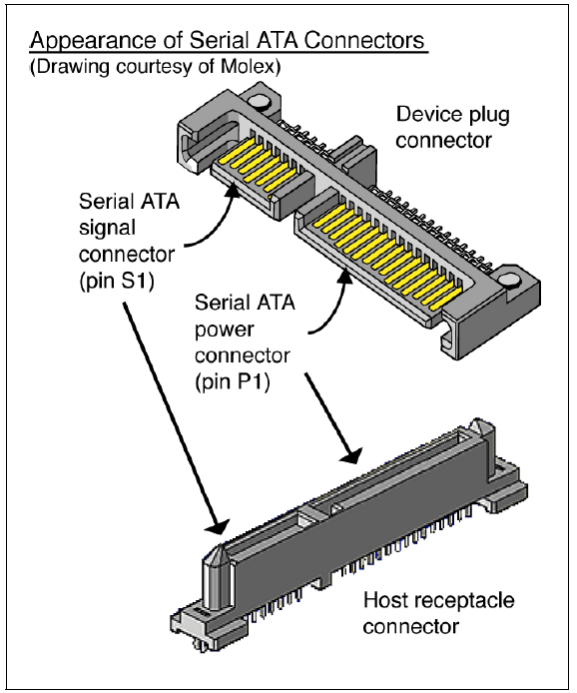
NVMe SSD

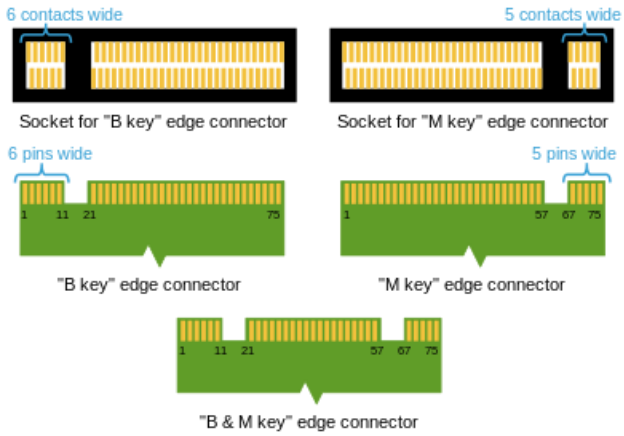
laptop

M.2 NVMe



MacBook Air





FTL

3가지 종류 ① Directed mapped FTL read-modify-write
② Log-structured FTL 새로운 공간에 쓰고(out of place) + mapping table + GC
③ DFTL mapping table이 너무 커서까 필요한 것만 caching

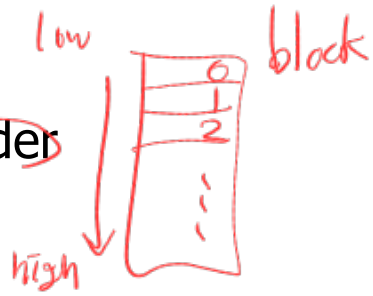
- FTL takes read and write requests on logical blocks and turns them into low-level read, erase, and program commands on the underlying physical blocks and physical pages
- Performance issues
  - Utilizing multiple flash chips in parallel
  - Reduce write amplification
    - total write issued to flash chips/total write issued by host

Garbage Collection in SSD
10MB + α / host(10MB) → WAF > 1
write amplification factor

- Reliability issues

limited P/E (life time)

  - Wear leveling
    - spread writes across the blocks of the flash as evenly as possible
    - ensure that all of the blocks of the device wear out at roughly the same time
  - Program disturbance
    - sequential-programming
      - program pages within an erased block in order
      - low page → high page



FTL: Direct Mapped

(logical block address) (physical page address)
LBA (host) → PPA (flash)

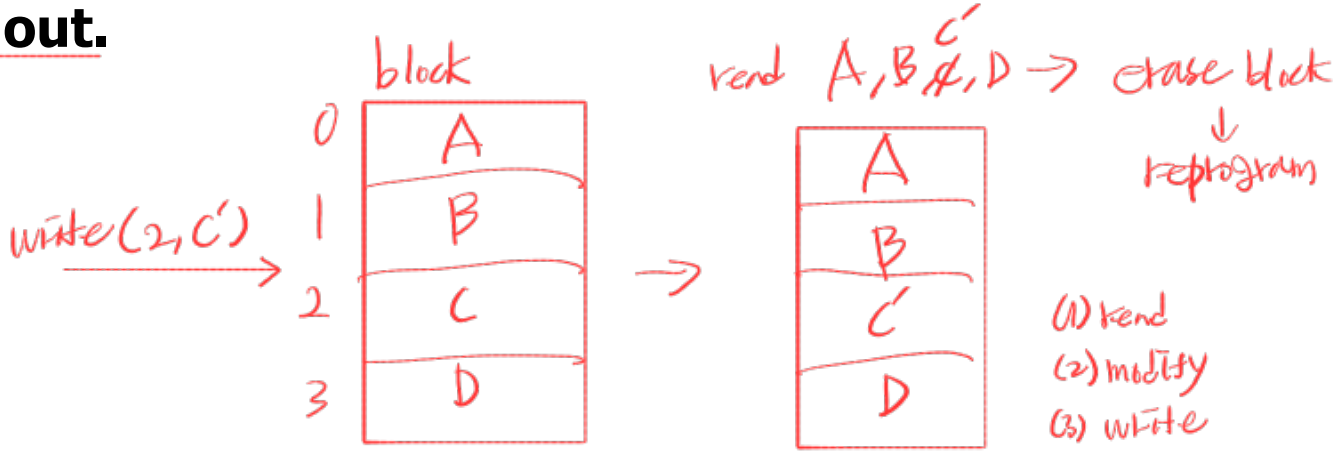
- **A write to logical page N**
  - Read in the entire block that page N is contained within
  - Erase the block
  - Program the old pages as well as the new one.
  - Read-modify-write approach

write할 page만 새로 new data로 program (read-modify-write)

- **Poor performance & high write amplification**

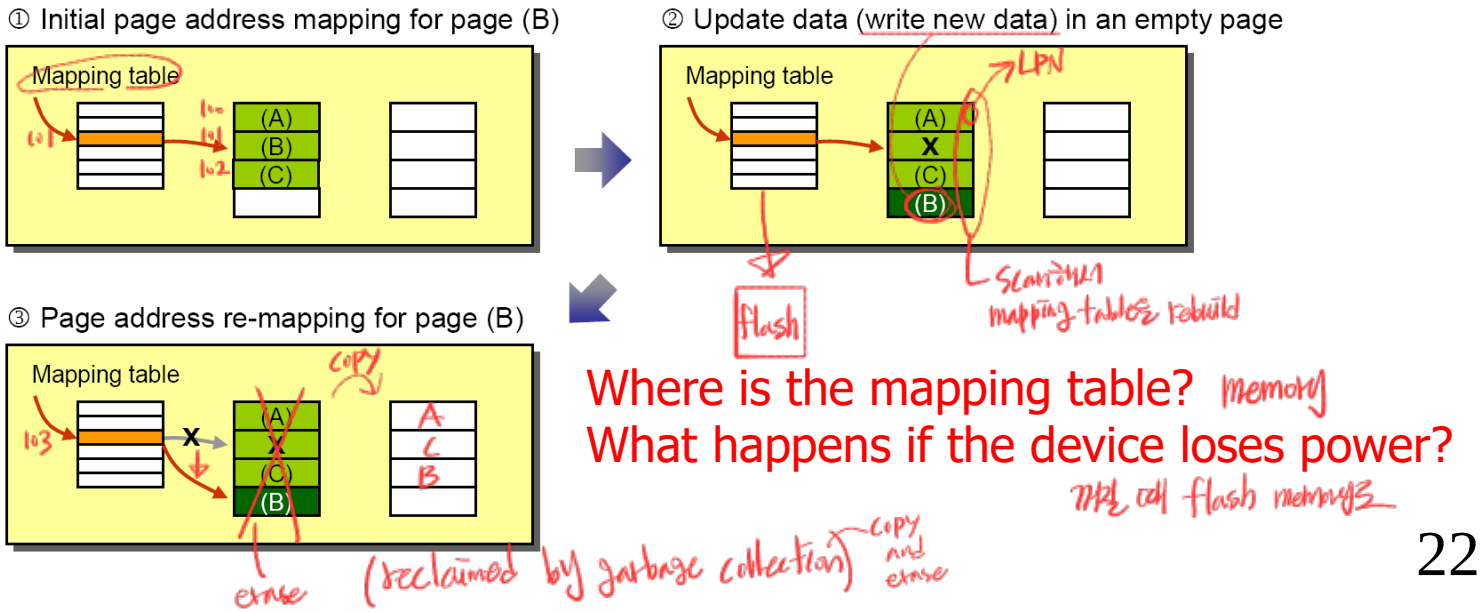
read all block / only 1 page → 4 page write

- **The physical blocks containing popular data will quickly wear out.**



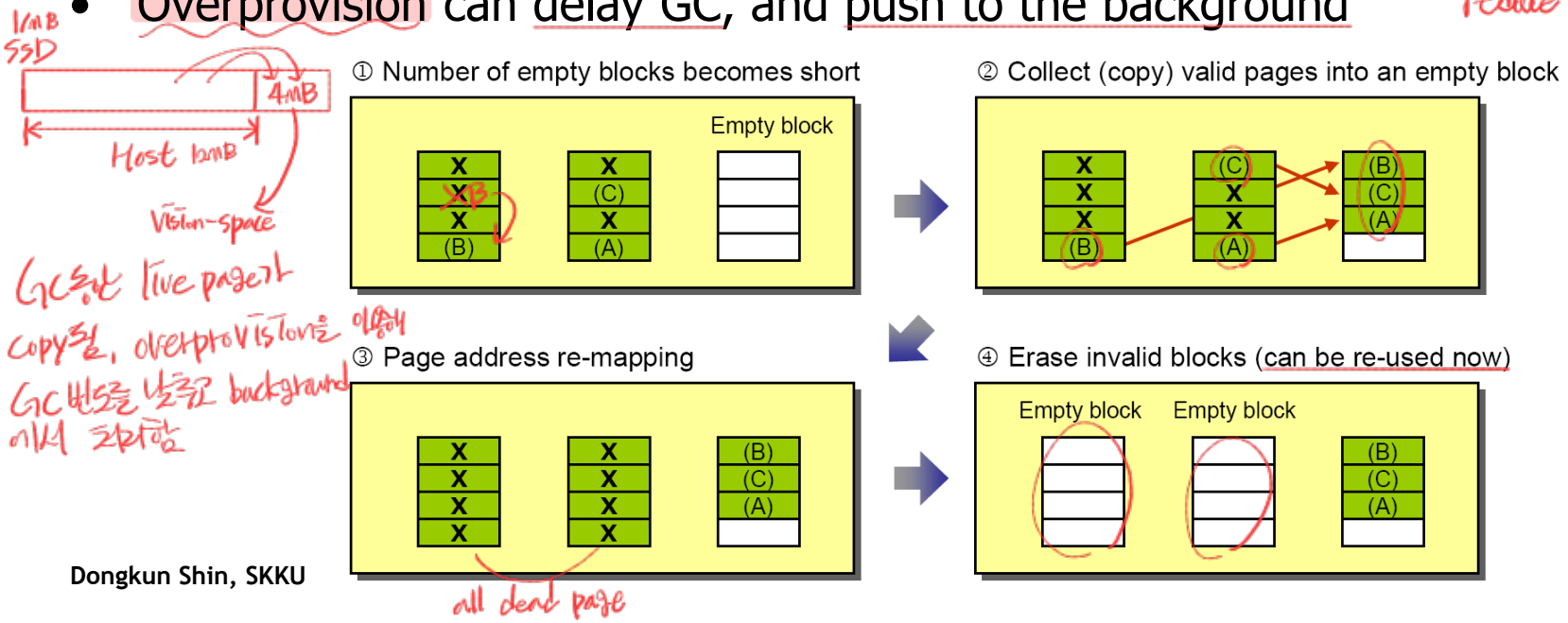
A Log-Structured FTL

- Upon a write to logical block N
  - The device appends the write to the next free spot in the currently-being-written-to block
  - no overwrite, out-of-place update write other place
  - To allow for subsequent reads of block N, the device keeps a mapping table, which stores the physical address of each logical block.
- Problems: GC & mapping table



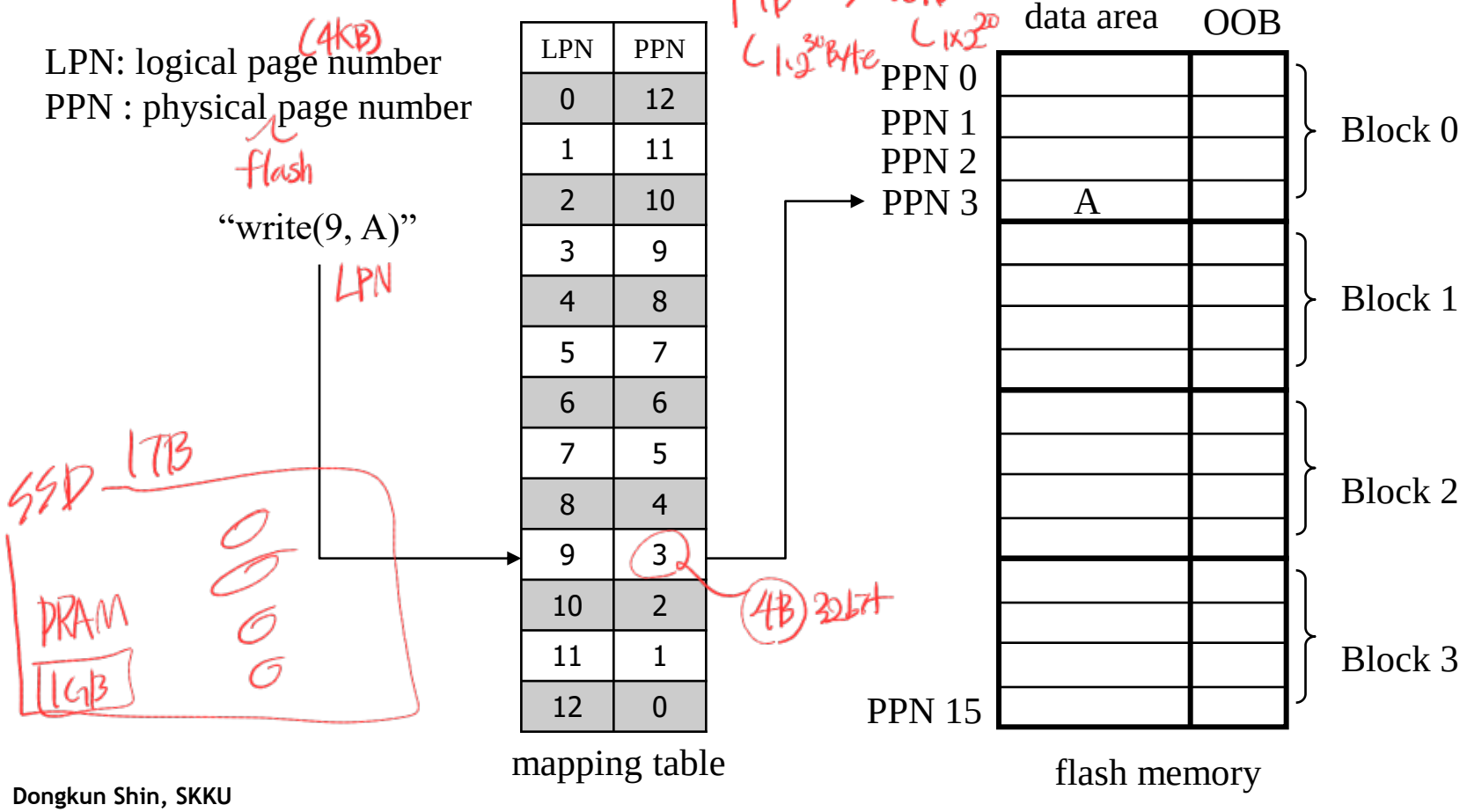
Garbage Collection

- dead-block reclamation to provide free space for new writes
  - find a block that contains one or more garbage pages
  - read in the live (non-garbage) pages from that block, write out those live pages to the log
  - (finally) reclaim the entire block for use in writing
- Must know
  - Whether each page is live or dead ➔ mapping table or bitmap
  - The number of live pages in each block for victim selection
- Overprovision can delay GC, and push to the background

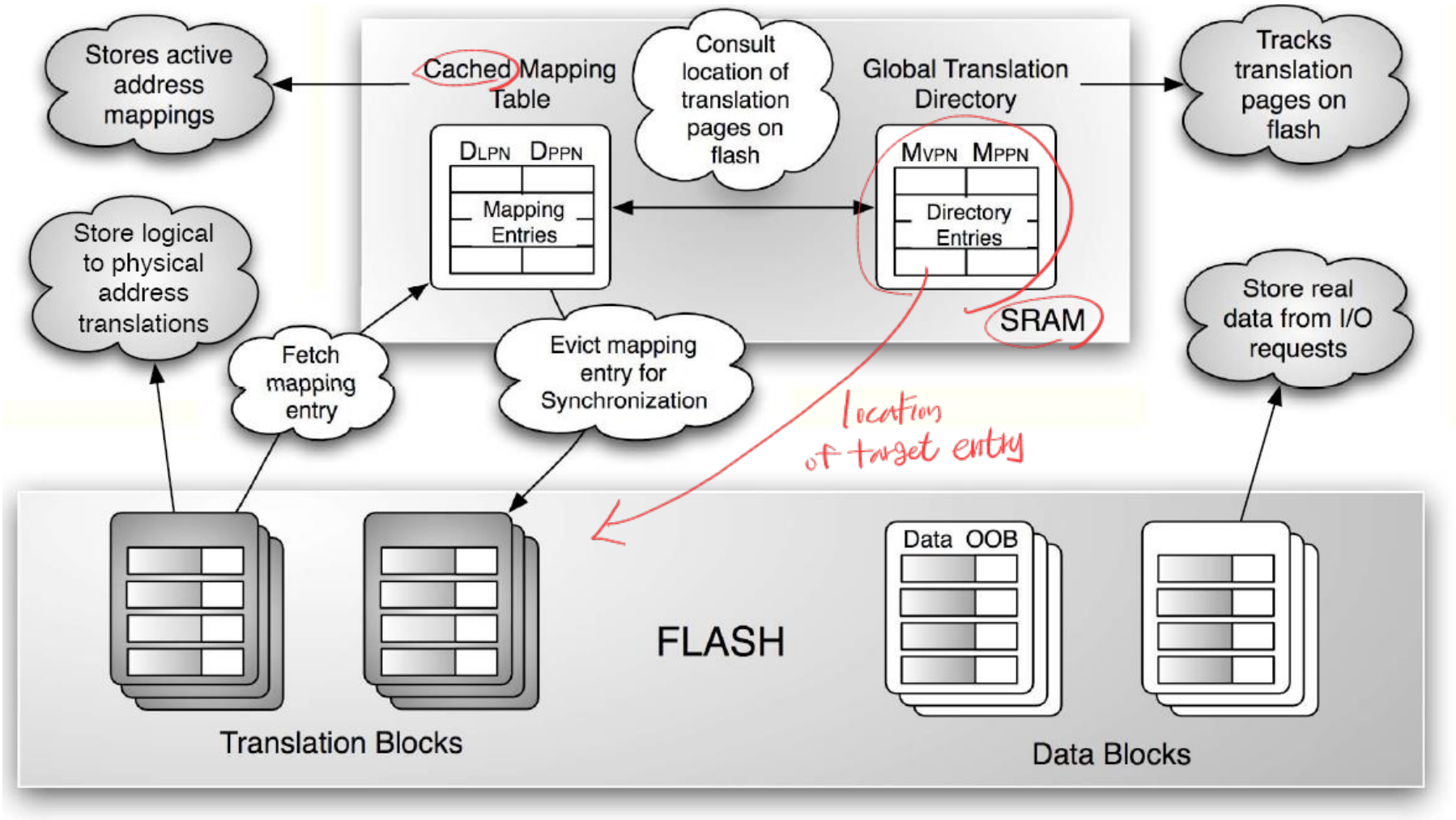


Mapping Table Size

- Page-level mapping
  - With a large 1-TB SSD, a single 4-byte entry per 4-KB page results in 1 GB of memory needed the device



DFTL caching



DFTL: A Flash Translation Layer Employing Demand-based Selective Caching of Page-level Address Mappings (ASPLOS'09)

SSD Performance

- Random I/O
  - Large difference between SSD and HDD
- Sequential I/O
  - Much less of a difference between SSD and HDD
  - HDD still a good choice
- SSD random read performance is not as good as SSD random write performance
  - Write buffer
- Performance difference between sequential and random in SSD
  - many of the file system techniques for HDDs are still applicable to SSDs

Device	Random Reads (MB/s)	Random Writes (MB/s)	Sequential Reads (MB/s)	Sequential Writes (MB/s)
Samsung 840 Pro SSD	103	287	421	384
Seagate 600 SSD	84	252	424	374
Intel SSD 335 SSD	39	222	344	354
Seagate Savvio 15K.3 HDD	2	2	223	223

Quiz

1. A status register in a device can be written by the OS to issue commands. (X)
2. DMA reduces CPU involvement in data transfer by offloading the task to a dedicated controller. (O)
3. Memory-mapped I/O uses the same address space for device registers as for regular memory. (O)
4. NAND flash memory supports overwriting data in-place without erasing. (X)
5. TLC flash stores fewer bits per cell compared to SLC. (X)
6. Log-structured FTLs write data out-of-place and update a mapping table. (O)
7. Page-level mapping in a 1TB SSD can require over 1GB of DRAM for the mapping table. (O)
8. DFTL reduces the memory footprint by caching only needed mapping entries. (O)
9. SSDs show much greater performance advantage over HDDs in sequential I/O than in random I/O. (X)
10. Write amplification in SSDs refers to writing more data to flash than was originally requested. (O)

Device Driver Implementation

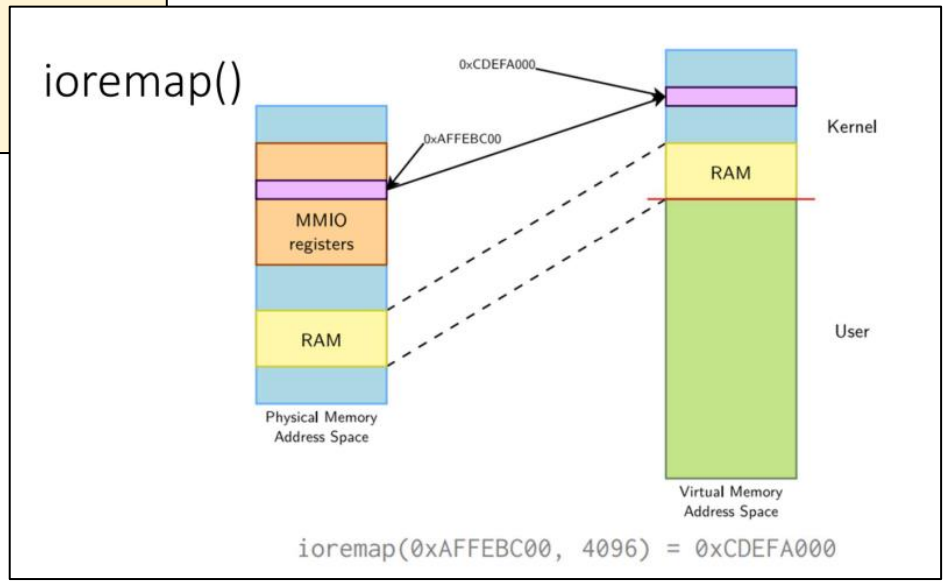
- **ioremap()**
  - Used by kernel drivers to access device memory (e.g., PCIe BARs).
  - Takes a physical address (e.g., 0xFE000000) and maps it into kernel virtual space.
  - Result must be accessed with readl(), writel(), etc.

```
void __iomem *mmio_base = ioremap(phys_addr, size);
```

```
// Suppose BAR0 = 0xAFFEBC00, size = 4 KB
mmio_base = ioremap(0xAFFEBC00, 0x1000);
if (!mmio_base) return -ENOMEM;

// Access register
writel(0x1, mmio_base + 0x04);
```

You should not directly access addresses returned by ioremap as if they were pointer to virtual memory address.
The read and write functions are defined to be ordered - Guarantee read/write ordering.



Device Driver Implementation

- **request_mem_region()**
  - Safe Reservation of MMIO Regions in Kernel
  - ensures that your driver exclusively reserves a physical memory range (typically MMIO registers) so that no other kernel driver will map or access it.

```
struct resource *request_mem_region(resource_size_t start,
                                    resource_size_t len,
                                    const char *name); // A string identifying your
                                                       //device (used in /proc/iomem).
```

```
resource_size_t start = pci_resource_start(pdev, 0);
resource_size_t len   = pci_resource_len(pdev, 0);

if (!request_mem_region(start, len, DEVICE_NAME)) {
    pr_err("Memory region busy\n");
    return -EBUSY;
}

void __iomem *mmio = ioremap(start, len);

iounmap(mmio);
release_mem_region(start, len);
```

Device Driver Implementation

- **dma_alloc_coherent()**
  - kernel API used to allocate memory for DMA
  - DMA-safe: accessible by both the CPU and a PCIe (or similar) device.
  - Physically contiguous: suitable for DMA hardware that cannot scatter-gather.
  - Uncached (or cache-coherent): ensures device and CPU see the same data.

```
void *dma_alloc_coherent(struct device *dev,
                         size_t size,
                         dma_addr_t *dma_handle,
                         gfp_t gfp);
Parameters:
• dev: pointer to the device (e.g., from &pdev->dev)
• size: size of memory to allocate (in bytes)
• dma_handle: output parameter to get the bus address for the
  device
• gfp: memory allocation flags (GFP_KERNEL, GFP_DMA32, etc.)
Returns:
• Kernel virtual address to the allocated memory
• Use dma_handle to program the device with the DMA physical
  address
```

Device Driver Implementation

- **remap_pfn_range()**
  - Used inside the driver's mmap() callback to expose physical memory to user space.
  - Takes a page frame number (PFN) = physical address >> PAGE_SHIFT.
  - Used when user programs want to access MMIO or DMA buffers directly.

physical memory를 user-space에 expose

```
int remap_pfn_range(struct vm_area_struct *vma,
                    unsigned long user_start,
                    unsigned long pfn_start,
                    unsigned long size,
                    pgprot_t prot);
```

```
static int my_mmap(struct file *filp, struct vm_area_struct *vma)
{
    unsigned long phys = 0xfe000000; // device MMIO start
    unsigned long pfn = phys >> PAGE_SHIFT;
    unsigned long size = vma->vm_end - vma->vm_start;

    return remap_pfn_range(vma, vma->vm_start, pfn,
                           size, vma->vm_page_prot);
}
```

Device Driver Implementation

- **request_irq()**
  - register an interrupt handler for a given IRQ line, enabling the driver to respond to hardware events (like DMA completion, packet arrival, etc.).

```
int request_irq(unsigned int irq,
                irq_handler_t handler,
                unsigned long flags,
                const char *name,
                void *dev_id);
```

```
static irqreturn_t my_irq_handler(int irq, void *dev_id) {
    pr_info("Interrupt received on IRQ %d\n", irq);
    // Acknowledge or clear the interrupt in hardware
    return IRQ_HANDLED;
}
…
int ret = request_irq(pdev->irq, my_irq_handler,
                      IRQF_SHARED, "my_pci_driver", pdev);
…
free_irq(pdev->irq, pdev);
```